

# Parallel and Distributed Computing

## Assignment 6

**Instructor:** Dr. Saif Nalband

**Course Code:** UCS645

---

## Section A: CUDA Device Query

### Objective:

Write and execute a CUDA program to extract detailed information about the GPU device, including architecture, memory, and execution limits.

### Key Parameters Extracted:

- GPU Name: Tesla T4
  - Architecture: Turing
  - Compute Capability: 7.5
  - Global Memory: 14.56 GB
  - Shared Memory: 49152 bytes
  - Constant Memory: 65536 bytes
  - Warp Size: 32
  - Max Threads per Block: 1024
  - Max Block Dimensions: (1024, 1024, 64)
- 

## Answers

### 1. GPU Architecture & Capability

The system uses an NVIDIA Tesla T4 GPU based on the Turing architecture with compute capability 7.5.

### 2. Maximum Block Dimensions

(1024, 1024, 64)

### 3. Maximum Threads (1D configuration)

Maximum threads =  $65535 \times 512 = 33,553,920$

### 4. Why not always use maximum threads?

Due to memory constraints, scheduling overhead, and possible performance degradation.

### 5. Limiting Factors

- Shared memory size
- Register usage
- Hardware limitations
- Kernel configuration

### 6. Shared Memory

Fast on-chip memory shared within a block.

Available: **49152 bytes**

### 7. Global Memory

Large but slower memory accessible to all threads.

Available: **14.56 GB**

### 8. Constant Memory

Read-only cached memory for all threads.

Available: **65536 bytes**

### 9. Warp Size

A warp consists of 32 threads executed simultaneously.

### 10. Double Precision Support

Yes, supported.

---

## Analysis

### Observation:

The GPU supports a large number of parallel threads, indicating strong computational capability.

**Inference:**

Execution follows SIMD-style parallelism (warp-based execution). Shared memory is significantly faster than global memory and should be used for performance-critical tasks.

**Conclusion:**

The Tesla T4 GPU offers efficient parallel computation with optimized memory hierarchy.

---

## Section B: Parallel Sum using CUDA

**Objective:**

Compute the sum of a large floating-point array using GPU parallelism.

---

**Execution Steps**

1. Allocate memory on GPU
  2. Transfer data from host to device
  3. Configure thread blocks and grid
  4. Launch CUDA kernel
  5. Synchronize execution
  6. Copy results back to host
  7. Free allocated memory
- 

**Working Explanation**

- Each thread computes its index and processes one element
  - `atomicAdd` ensures safe accumulation into a shared variable
  - Synchronization ensures correctness before copying results
- 

**Output**

Sum of array = 1048576.00 (Expected: 1048576)

---

## Inference

The program demonstrates effective parallel execution, significantly improving computation speed over sequential approaches.

### Limitation:

Using `atomicAdd` introduces serialization, which can reduce performance for very large datasets.

---

## Conclusion

CUDA efficiently distributes workload across threads, enabling fast computation of large datasets.

---

## Section C: Matrix Addition in CUDA

### Objective:

Perform parallel addition of two large matrices.

---

### Approach

- Use a 2D grid and block structure
  - Each thread computes one matrix element
  - Map thread indices to matrix coordinates
- 

### Execution Details

- Matrix Size:  $1024 \times 1024$
  - Block Size:  $16 \times 16$
  - Grid Size: Calculated dynamically
- 

## Computation Analysis

### 1. Floating Point Operations

None (integer operations only)

### **Total Operations:**

$1024 \times 1024 = 1,048,576$  additions

---

### **2. Global Memory Reads**

Each thread reads 2 values

Total Reads =  $2 \times N \times N =$

**2,097,152 reads**

---

### **3. Global Memory Writes**

Each thread writes 1 value

Total Writes =  $N \times N =$

**1,048,576 writes**

---

## **Output**

Matrix addition completed successfully

Sample output:  $C[0][0] = 0$

---

## **Inference**

Each thread independently computes one element, making the algorithm highly parallelizable.

---

## **Final Conclusion**

Matrix addition is inherently parallel, and CUDA effectively accelerates computation by distributing work across thousands of threads.